```
});
const { textStream } = await streamText({.
  model: model('gemini-2.0-flash'),
  prompt: 'Generate 400 words of random content for me please',
});

for await (const textPart of textStream) {.
  process.stdout.write(textPart);
}
```

The streamText function is called with an object containing the model (more on this later) and the prompt text.

The code uses a for await...of loop to iterate over the textStream async iterable. This loop allows asynchronous iteration over the text stream, handling each chunk of text as it becomes available.

In this example, we import the `createGoogleGenerativeAI` function from `@ai-sdk/google` to create a Google Gemini model and the `streamText` function from the `ai` package. We then call the `streamText` function with the desired model and prompt, and it returns a `textStream` object.

The `textStream` is an `async` iterable, which means you can use a `for await...of` loop to iterate over the text parts as they are generated. Each `textPart` represents a chunk of the generated text, and you can process or display it in real time.

To understand what an `async` iterable is, look at a simple example that generates a sequence of numbers with a delay between each number. I'll use this example to illustrate the concept of an `async` iterable and how it can be consumed using a `for await...of` loop.

Listing 3.4 An `async` iterable example

```
async function* getOneToThree() {.
  for (let i = 1; i <= 3; i++) {
    await new Promise(resolve =>
              setTimeout(resolve, 1000));
    yield i;
  }
}
```

An async generator function is declared using the async function* syntax.

The await keyword is used to pause the execution of the async generator function until the promise is returned.

The yield keyword is used to yield the current value of i to the consumer of the async iterable.

In this code, we define an `async` generator function called `getOneToThree`. Inside the function, we use a `for` loop to iterate from 1 to 3. For each iteration, we introduce a delay. After the delay, we use the `yield` keyword to yield the current value of `i`.

When the generator function completes its execution (in this case, after yielding 3), it behaves in a specific way:

1. The generator function automatically returns a promise that resolves to `{ done: true, value: undefined }`.
2. Any subsequent calls to `next()` on the generator object will return this same resolved promise.
3. The `for...of` loop or any other consuming iterator will terminate naturally.